

Dark Ascent Code Doc

1. Game Overview

The Dark Ascent is a first-person action dungeon crawler set in a dark industrial world. Players navigate procedurally generated levels, fight enemies using melee combat with a combo system, collect mutations that augment their abilities, and progress through increasingly challenging floors.

2. Player System

FirstPersonController.cs (Singleton)

The primary player controller, built on Unity's CharacterController component. Handles all player input, movement, gravity, combat, and animation.

Movement

- WASD movement with configurable speed and acceleration
- Gravity system with ground detection
- Dodge mechanic: speed boost on Left Shift, with cooldown timer

Combat

- 3-hit melee combo system (Attack1, Attack2, Attack3) on left click
- Heavy attack on right click with separate cooldown
- Attack distance and damage configurable via inspector
- Layer mask filtering for valid attack targets
- Animation-driven attack timing with cooldowns between swings

PlayerManager.cs (Singleton)

Central stats manager for the player. Tracks health (max 5), attack damage, movement speed, and acceleration. Provides TakeDamage(), GetAttackDamage(), GetMoveSpeed(), GetAcceleration(), GetHealth(), and GetMaxHealth() methods. Applies mutation damage multipliers to the base attack value.

Supporting Scripts

- **PlayerHurtbox.cs** - Implements IHittable. Receives damage from enemies via trigger collisions. Activates DamageIndicator visual flash.
- **PlayerHitbox.cs** - Player's melee attack collider for hit detection.
- **PlayerLook.cs** - Mouse look and aiming. Handles camera rotation on both axes.
- **HeadBobController.cs** - Camera bobbing effect during player movement for immersion.
- **CameraManager.cs** (Singleton) - Manages 4 cameras: gameplay, menu, logbook, mutation inventory. Switches active camera based on game state.

3. Enemy AI System

All enemies use the UnityHFSM library for hierarchical finite state machines. Each enemy type has a unique FSM configuration with shared and specialized states. Enemies use NavMesh for pathfinding and sphere collider sensors for player detection.

Base Architecture

- **Enemy.cs** - Base class. Holds the `StateMachine<EnemyState, StateEvent>` instance. Requires `Animator` and `NavMeshAgent`. Provides `RunCoroutine()` for state coroutines.
- **EnemyStateBase.cs** - Abstract base for all states. Manages exit timing, animation triggers, `NavMesh` agent control (speed, stopping). Provides `onEnter/onLogic/onExit/canExit` callbacks.
- **EnemyManager.cs** - Enemy health and stats. Handles `TakeDamage()`, `Die()`, and loot drops via `WeightedRandomList`.
- **PlayerSensor.cs** - Sphere collider trigger. Fires `OnPlayerEnter(Transform)` and `OnPlayerExit(Vector3 lastKnownPos)` events.
- **EnemyState.cs** - Enum: `Idle`, `Patrol`, `Chase`, `Attack`, `Dash`, `Leap`, `Spore`, `Gather`, `LegAttack`.

Enemy Types

Butcher (ButcherAI.cs)

Aggressive melee enemy. Has chase, patrol, and dash-closing abilities.

- States: `Idle` -> `Patrol` -> `Chase` -> `Attack`, with `Dash` on 21s cooldown
- Sensors: `ChasePlayerSensor`, `MeleeRangeSensor`, `DashRangeSensor`
- Attack cooldown: 2 seconds between strikes

Crawler (CrawlerAI.cs)

Fast, mobile enemy that leaps at the player from range.

- States: `Idle` -> `Chase` -> `Leap` (10s cooldown)
- Sensors: `ChasePlayerSensor`, `MeleeRangeSensor`

Mushroom (MushroomAI.cs)

Ranged enemy that throws spore projectiles using object pooling.

- States: `Idle` -> `Patrol` -> `Chase` -> `Spore`, with `Gather` recovery state
- Sensors: `ChasePlayerSensor`, `ProjectileRangeSensor`
- Spore projectile: physics-based, 1s auto-destroy, damages on contact

Bladelegs (BladelegsAI.cs)

Multi-limb enemy with leg-based attack patterns.

- States: `LegAttack`, `LegDash`
- Specialized melee attacks with unique animation set

4. Combat System

Combat uses an interface-based damage system for extensibility. Any object implementing `IAttacker` can deal damage

to any object implementing IHittable.

Interfaces

- **IAttacker** - Implemented by objects that deal damage. Provides a DamageInfo struct.
- **IHittable** - Implemented by objects that receive damage. Exposes GetHit(DamageInfo) method.

Damage Flow

1. Player attacks trigger a raycast or collider check within attack distance
2. If a valid target is found (matching layer mask), DamageInfo is constructed
3. Target's IHittable.GetHit() is called with the damage amount
4. For enemies: EnemyManager.TakeDamage() reduces HP, triggers death and loot on 0
5. For player: PlayerManager.TakeDamage() reduces hearts, triggers DamageIndicator flash

Key Scripts

- **DamageInfo.cs** - Struct carrying the damage amount between attacker and target.
- **SlashAttack.cs** - Rotating mesh projectile for visual attack feedback. 40ms lifetime.
- **Spore.cs** - Mushroom projectile. Implements IAttacker. Uses Rigidbody force, 1s auto-destroy.
- **EnemyHitbox.cs** - Trigger collider on enemy attack animations. Deals damage on overlap with player.
- **DamageIndicator.cs** - Screen flash effect when the player takes damage.

5. Mutation System

Mutations are collectible power-ups that players equip to 7 body slots, modifying combat stats. The system uses ScriptableObjects for data, a singleton manager for inventory logic, and dedicated UI panels for equipment management.

Data Layer

- **MutationInfo.cs** (ScriptableObject) - Defines a mutation: name, icon, description, allowedSlots[], damageMultiplier, rangeMultiplier. Created via Assets > Create > Mutations > Mutation Info.
- **MutationSlotType.cs** - Enum: Head, Chest, Heart, LeftArm, RightArm, LeftLeg, RightLeg (7 slots).

Inventory Management

- **MutationInventoryManager.cs** (Singleton) - Tracks collectedMutations list and equippedMutations dictionary (keyed by slot). AddMutation() auto-equips to first compatible empty slot. EquipMutationToSlot() and UnequipMutationFromSlot() for manual management. Recalculates player stats on any change.
- **PlayerMutationHandler.cs** (Singleton) - Applies mutation multipliers (damage, range) to player stats. ApplyMutation() adds multipliers, ResetMutationEffects() clears them.

Pickup Flow

1. Enemy dies -> EnemyManager.Die() rolls WeightedRandomList loot table
2. MutationPickup spawns with rotating visual and UI prompt (icon, name, slot, 'Press E')
3. Player presses E -> MutationInventoryManager.AddMutation() called
4. Auto-equip logic finds first compatible empty slot

5. PlayerMutationHandler recalculates damage/range multipliers

6. Level Generation

The game uses procedural level generation to create dungeon layouts at runtime. The system places grid-aligned sections (rooms, hallways) in random configurations.

Key Scripts

- **LevelGenerationManager.cs** (Singleton) - Orchestrates generation. Config: numberOfSections (default 10). Maintains sectionGridDict for spatial tracking. GenerateLevel() kicks off the process, spawning sections until the target count is reached.
- **LevelSection.cs** - Base section class. Each section occupies a grid cell and has possibleDirections (forward, back, left, right). Uses raycasting to detect which directions are available. SpawnNewSections() instantiates random adjacent sections.
- **StartingSection.cs** - The initial spawn room. Inherits LevelSection behavior.
- **HallwaySection.cs** - Corridor section connecting rooms.
- **Directions.cs** - Utility providing AllDirections array and LocalizePossibleDirections() for converting world-space directions to local section space.

Section Prefabs

- Spawn Room - Starting area
- Basic Room Whitebox - Standard combat room
- Hallway Whitebox - Connecting corridor
- Boss Room - End-of-floor encounter
- Inaccessible Path Block - Dead-end filler

7. UI & Menu System

The UI is split across multiple canvases managed by singletons. Camera switching provides distinct viewpoints for gameplay, menus, logbook, and mutation inventory.

Menu Management

- **MenuManager.cs** (Singleton) - Controls logbookCanvas, mutationInventoryCanvas, mutationSlotPanel, mutationDetailPanel. Keyboard: Q = logbook, Tab = mutations, Escape = close all.
- **UIManager.cs** (Singleton) - Alternative UI controller for logbook and mutation panels. AnyUIOpen() checks if any overlay is active.

Mutation Inventory UI

- **MutationInventoryUI.cs** - Full-screen equipment panel with 7 MutationSlotUI references. Camera focus points per slot for 3D model rotation. Detail panel shows mutation name, icon, and description.
- **MutationSlotUI.cs** - Individual slot widget. Displays equipped mutation icon or empty state.
- **MutationDetailUI.cs** - Expanded info panel when selecting a mutation.
- **RemoveConfirmationPanel.cs** - Confirmation dialog for unequipping mutations.

Logbook

- **LogbookManager.cs** (Singleton) - Enemy database with EnemyInfo entries. Panels: enemyListPanel, mutationPanel, mutationDetailPanel. Tracks which enemies have been encountered (isUnlocked). -
- EnemyInfo.cs** - Serializable class: enemyName, enemyIcon, enemyModelPrefab, description, isUnlocked, possibleMutations[].
- **EnemyEntry.cs / EnemyListPopulator.cs** - List item widget and population logic.

HUD

- **HealthUIManager.cs** - Heart-based HP display. Spawns heart prefabs, syncs full/empty state with PlayerManager.GetHealth().
- **GameOverUI.cs** (Singleton) - Shows on player death. Freezes time (timeScale = 0). Retry reloads scene, Quit exits application.

8. Art & Assets

Animations

- **Player** - 1st Person controller with Idle, Walk, Dash, Attack Swing, Backswing. Separate upper/lower body masks for blending.
- **Butcher** - Idle, Walk, Slam, Slash animations with dedicated Animator controller.
- **Crawler** - Idle, Walk, Jump animations.
- **Mushroom** - Idle, Walk, Throw, Object Pickup animations. Includes Main Menu Easter Egg animation.
- **Bladelegs** - Walk, Idle, Dash, Attack, Blade Legs special animation.

Models

- 4 enemy FBX models: Butcher, Crawler, Mushroom, Bladelegs
- Player character model with fixed origin variant
- Weapons: main character pipe, longer pipe
- World models: Barrel, Corner, Elevator-Door, Elevator-Wall, Generator, Lantern, Wall-Gears, Window-Corner
- Steampunk environment assets: SteamPunkAssets, SteamPunkWall, SteamPunkMapLayout

Shaders

- **Dithering Shader Graph** - Retro dithering visual effect using Bayer matrix (bayer8tile4.png). Gives the game its signature stylized look.
- **Slash Shader Graph** - Visual effect shader for melee attack slashes.

Materials

- Flat Colours: Solid color materials for whitebox/placeholder assets
- Textured Materials: Full PBR materials for production models
- MaterialImages: Organized texture libraries (Brick, CorrodedMetal, Floor, Generator, Marble, Metals, Table, WoodPlanks)

9. Scenes

- **Main Menu** (A.Scenes/Main Menu/) - Title screen with lighting bake data.
- **Vertical Slice Build** (A.Scenes/Vertical Slice Build/) - Primary playable scene with 2 NavMesh surfaces for enemy AI pathing.
- **Developer Test Scenes** - Per-developer folders (Evan, Luke, Nikita, Peter, Arthur) for isolated feature testing. Many cleaned up during refactoring.

10. Singletons & Managers

The project uses 11 singleton managers for global state access:

Singleton Responsibility

PlayerManager Player health, stats, damage, death events FirstPersonController Player movement, input, combat, animation CameraManager Switches between 4 cameras based on game state GameOverUI Death screen display, time freeze, retry/quit MutationInventoryManager Mutation collection, equipment, stat recalculation PlayerMutationHandler Applies mutation stat multipliers to player LevelGenerationManager Procedural level creation and section tracking MenuManager UI canvas and panel management, keyboard shortcuts UIManager Alternative UI controller for overlays LogbookManager Enemy database, unlock tracking, logbook panels HealthUIManager Heart-based health HUD synchronization

11. Key Interfaces & Enums

Interfaces

- **IAttacker** - Contract for damage-dealing objects. Exposes DamageInfo.
- **IHittable** - Contract for damage-receiving objects. Exposes GetHit(DamageInfo).

Enums

- **EnemyState** - Idle, Patrol, Chase, Attack, Dash, Leap, Spore, Gather, LegAttack
- **StateEvent** - Triggers for FSM transitions between enemy states
- **MutationSlotType** - Head, Chest, Heart, LeftArm, RightArm, LeftLeg, RightLeg
- **SectionType** - Categorization enum for level generation section prefabs